SafeTest-Dev | Binary Security Research

SECURITY ANALYSIS REPORT · CONFIDENTIAL

Lab05: DLL Key XOR Decoding

Reverse Engineering — Two-Stage Key Bypass · XOR Encoded DLL Key · hexor Solver

Report Date	April 2026
Author	Michael.A | SafeTest-Dev
Classification	Confidential – Authorized Use Only
Target Binary	loader.exe (Stage 1) + payload.dll (Stage 2)
Format	PE32+ · x86-64 · Stripped · Windows Binary
Analysis Type	Static Disassembly · Radare2 · XOR Brute Force · hexor
Tool	Radare2 (r2) · hexor (Crystal — custom XOR decoder)
Platform	Arch Linux + Wine
Stages	Stage 1: key123 → FLAG{STAGE1_PASS} | Stage 2: AI_SEC! → FLAG{STAGE2_DLL_PASS}

Findings Overview

ID	Finding	Component	Impact	Severity
ALC-01	XOR-Encoded Key in .text Section	movabs @ 0x1db60140c	Static Key Extraction	HIGH
ALC-02	Single-Byte XOR Brute-Forceable in O(255)	XOR loop 0x1db601440	Complete Key Recovery	CRITICAL
ALC-03	No Anti-Analysis in DLL	payload.dll .text section	Unrestricted Static Access	HIGH

> ■ *This documentation is prepared exclusively for defensive security research and educational purposes within the SafeTest-Dev framework. All techniques are documented for defensive understanding and incident response. DO NOT execute binaries on production or uncontrolled systems.*

1. Executive Summary

This report presents the security analysis of **Lab05** — the direct continuation of Lab04. The same target binary **loader.exe** (Secure Loader v2.1) is used, but with an additional challenge: after bypassing Stage 1, the binary loads a secondary payload (**payload.dll**) that presents a second key prompt. This DLL key is XOR-encoded and embedded as a 64-bit hex constant in the disassembly.

Three vulnerabilities were identified. The critical finding (ALC-02) demonstrates that single-byte XOR has a keyspace of exactly 255 values — exhausted instantly by hexor, a purpose-built Crystal tool that filters readable output automatically. Both flags were captured: FLAG{STAGE1_PASS} and FLAG{STAGE2_DLL_PASS}.

2. Target Overview

2.1 Binary Identification

Field	Value
Stage 1 Binary	loader.exe — PE32+, x86-64, Stripped to external PDB
Stage 2 Binary	payload.dll — PE32+, x86-64, Stripped
Format	PE32+ executable for MS Windows (console)
Architecture	AMD x86-64
Linkage	Stripped to external PDB — no embedded symbols
Execution Platform	Arch Linux + Wine
Analysis Tool	Radare2 (r2 -A)
Custom Tool	hexor — Crystal-language XOR brute force decoder

2.2 Two-Stage Architecture Overview

Stage	Binary	Key	Method	Flag
Stage 1	loader.exe	key123	Direct key supply (recovered from Lab04)	FLAG{STAGE1_PASS}
Stage 2	payload.dll	AI_SEC!	XOR decode 0x335157414d5b53 with key 0x12	FLAG{STAGE2_DLL_PASS}

3. Stage 1 Recap & Stage 2 Discovery

3.1 Stage 1: key123 (From Lab04)

In Lab04, loader.exe was analyzed via static disassembly and patched using NOP injection to bypass the first key validation. In Lab05, Stage 1 is approached differently — the correct Stage 1 key **key123** is supplied directly (recovered via prior analysis), which grants access and triggers payload.dll to load for Stage 2.

```
$ wine loader.exe run
r600 device_uuid output is based on invalid pci bus info.
== Secure Loader v2.1 ==
Enter key: key123
[+] Access granted
FLAG{STAGE1_PASS}
[*] Enter DLL key: randomkeyfortest
[-] DLL Wrong key
```

Stage 2 Discovery: After FLAG{STAGE1_PASS} is printed, the binary loads payload.dll and presents a second prompt: **[*] Enter DLL key:**. This is a separate validation implemented inside the DLL itself — a new target requiring independent analysis.

4. Phase 1: Static Analysis of payload.dll

4.1 Function Mapping with afl

payload.dll is opened in Radare2 for static analysis. The key target is **sym.payload.dll_run** — the exported function called by loader.exe when the DLL is loaded. This function implements the Stage 2 key validation (150 instructions).

```
[0x1db6011f0]> afl
0x1db6011f0  13  208  entry0
0x1db602280   1    6  fcn.1db602280
0x1db601000  30  474  section..text
0x1db601400   5  150  sym.payload.dll_run    <- TARGET
0x1db602320   1   83  fcn.1db602320
0x1db601810  54  887  fcn.1db601810
0x1db601570  13  158  fcn.1db601570

; Imports identified:
sub.api_ms_win_crt_string_l1_1_0.dll_strcmp
sub.api_ms_win_crt_string_l1_1_0.dll_strncmp
sub.api_ms_win_crt_stdio_l1_1_0.dll___stdio_common_vfscanf
sub.api_ms_win_crt_stdio_l1_1_0.dll_puts
```

4.2 Disassembly of sym.payload.dll_run

Seeking to sym.payload.dll_run and running pdf reveals the complete Stage 2 validation logic. The critical finding is at **0x1db60140c**: a movabs instruction loading a 64-bit hex constant into RAX. Radare2 annotates this constant with **"S[MAWQ3"** — the XOR-encoded DLL key.

```
[0x1db601400]> pdf
150: sym.payload.dll_run (int64_t arg2, int64_t arg3, int64_t arg4);
   '- args(rdx, r8, r9) vars(2:sp[0x70..0x78])
0x1db601400  53          push rbx
0x1db601401  4883ec70    sub rsp, 0x70
0x1db601405  488d0d442c  lea rcx, str.__Enter_DLL_key:
             ; "[*] Enter DLL key: "

; ■■■ KEY FINDING ■■■
```

5.2 Core Logic: parse_hex() — Hex String to Bytes (Little Endian)

Converts the input hex string to a byte array. Automatically handles the 0x prefix, pads odd-length strings, and reverses the byte order to account for little-endian storage in x86-64 memory.

```
def parse_hex(input : String)
  hex = input.starts_with?("0x") ? input[2..] : input
  hex = "0" + hex if hex.size.odd?
  bytes = [] of UInt8
  hex.chars.each_slice(2) do |pair|
    bytes << pair.join.to_i(16).to_u8
  end
  bytes.reverse!  # little endian correction
  bytes
end
```

5.3 Core Logic: xor_decode() — Single-Byte XOR Decode

XORs every byte in the array with the given key and joins the results into a string. Self-inverse property: applying the same key twice returns the original bytes.

```
def xor_decode(bytes : Array(UInt8), key : Int32)
  bytes.map { |b| (b ^ key).chr }.join
end
```

5.4 Core Logic: readable?() — Printability Filter

Filters out noise from brute force results. A decoded string is considered readable only if more than 85% of its characters fall within the printable ASCII range (0x20–0x7E). This dramatically reduces false positives when iterating through 255 XOR candidates.

```
def readable?(s : String)
  return false if s.size == 0
  printable = s.count { |c| c.ord >= 32 && c.ord <= 126 }
  printable > s.size * 0.85
end
```

5.5 Core Logic: brute_xor() — Brute Force Mode (--brute)

Iterates through all 255 possible single-byte XOR keys (0x01–0xFF), decodes the input with each, and prints only results that pass the printability filter. The matching key is identified visually from the output.

```
def brute_xor(bytes)
  puts "\n[+] Brute forcing XOR keys..."
  (1..255).each do |k|
    decoded = xor_decode(bytes, k)
    if readable?(decoded)
      puts "[key 0x#{k.to_s(16)}] => #{decoded}"
    end
  end
end
```

5.6 Usage

```
# Brute force all 255 XOR keys against hex constant from disassembly
hexor 0x335157414d5b53 --brute

# Manual decode with specific key
hexor 0x335157414d5b53 0x12

# Output:
[+] XOR key : 0x12
```

```
[+] Result  : AI_SEC!
```

6. Phase 2: XOR Brute Force Decoding

6.1 Running hexor --brute

```
$ hexor 0x335157414d5b53 --brute

  _   _ ________  ______   ______
 | | | |/ / ____| / __ \ \/ / __ \
 | |_| | /| |__  | | | \  /| | |
 | _ < | __| | | | |/ | | | |
 | | | . \| |____| |__| |  | |__| |
 |_| |_|\_\______| \____/ |_|\_\____/
            HEX XOR DECODER
 ======================================

[+] Brute forcing XOR keys...
[key 0x1]  => RZL@VP2
[key 0x2]  => QYOCUS1
[key 0x3]  => PXNBTR0
...
[key 0x11] => BJ\PF@"
[key 0x12] => AI_SEC!    <- MATCH
[key 0x13] => @H^RDB
...
```

XOR Key Identified: Hex Constant: 0x335157414d5b53 — XOR Key: 0x12 (decimal: 18) — Decoded Value: **AI_SEC!**. Key 0x12 is the only result that produces a clean, readable ASCII string matching an expected key format. All other candidates produce noise.

6.2 Manual XOR Verification

The hex constant 0x335157414d5b53 in little-endian byte order decodes to AI_SEC! when each byte is XORed with 0x12:

Encoded Byte	XOR Key	Result Byte	ASCII Char
0x53	0x12	0x41	A
0x5B	0x12	0x49	I
0x4D	0x12	0x5F	_
0x41	0x12	0x53	S
0x57	0x12	0x45	E
0x51	0x12	0x43	C
0x33	0x12	0x21	!

7. Phase 3: Verification & Final Result

7.1 Supplying AI_SEC! as DLL Key

```
$ wine loader.exe run
r600 device_uuid output is based on invalid pci bus info.
== Secure Loader v2.1 ==
Enter key: key123
[+] Access granted
FLAG{STAGE1_PASS}
[*] Enter DLL key: AI_SEC!
```

```
[+] DLL Access granted
FLAG{STAGE2_DLL_PASS}
```

7.2 Technical Summary

Parameter	Detail
Target (Stage 1)	loader.exe — PE32+, x86-64, Stripped
Target (Stage 2)	payload.dll — PE32+, x86-64, Stripped
Stage 1 Key	key123 (recovered from Lab04 analysis)
Analysis Function	sym.payload.dll_run (150 instructions)
Encoded Constant	0x335157414d5b53 @ 0x1db60140c (movabs rax)
XOR Key	0x12 (decimal: 18)
Decoded Key	AI_SEC!
Decode Method	hexor --brute (255-candidate single-byte XOR)
Flag 1 Captured	FLAG{STAGE1_PASS}
Flag 2 Captured	FLAG{STAGE2_DLL_PASS}
Status	SUCCESS — Both stages completed

8. Vulnerability Findings

ALC-01	HIGH
Component	payload.dll — .text section @ 0x1db60140c
Method	Static disassembly (r2 pdf) — movabs instruction extraction
Impact	Complete 7-byte encoded key extracted statically — no execution required

The DLL key is stored as a 7-byte XOR-encoded hex constant (0x335157414d5b53) directly in the .text section via a movabs instruction. While this provides minimal obfuscation compared to plaintext storage, it is trivially extractable through static disassembly — Radare2 even annotates the raw string S[MAWQ3 directly in the output.

ALC-02	CRITICAL
Component	XOR decode loop @ 0x1db601440–0x1db60144f — single-byte XOR
Method	hexor --brute mode — 255-candidate exhaustive XOR decode + printability filter
Impact	Correct DLL key recovered in milliseconds with zero prior knowledge of key value

Single-byte XOR encryption has a keyspace of exactly 255 meaningful values. A brute force approach — as implemented in hexor — exhausts the entire keyspace instantly. The printability filter (85%+ printable ASCII) automatically isolates AI_SEC! from 254 noise results. The XOR constant 0x12 is also directly visible in the static disassembly at 0x1db601440.

ALC-03	HIGH
Component	payload.dll — no anti-debug, anti-tamper, or obfuscation
Method	Static analysis — absence of standard protection mechanisms
Impact	Unrestricted static analysis — XOR logic and constants fully readable

The DLL contains no anti-debug, anti-tamper, or obfuscation mechanisms. The XOR decode loop is clearly visible in static disassembly at 0x1db601440–0x1db60144f. The XOR constant (0x12) is hardcoded and immediately visible in the disassembly without any dynamic resolution, obfuscation, or protection layer.

9. Risk Assessment

ID	Finding	Likelihood	Impact	Overall
ALC-01	XOR-Encoded Key in .text Section	High	High	HIGH
ALC-02	Single-Byte XOR Brute-Forceable in O(255)	High	Critical	CRITICAL
ALC-03	No Anti-Analysis in DLL	Medium	High	HIGH

10. Recommendations

#	Recommendation	Impact
1	Use strong symmetric encryption (AES) instead of single-byte XOR for embedded keys	Eliminates brute force feasibility
2	Derive keys at runtime from multiple sources — never hardcode as a static constant	Prevents static extraction from disassembly
3	Implement anti-debug and anti-tamper in DLL — detect r2 / Wine environment	Raises analysis bar significantly
4	Validate DLL integrity before loading — check signature or hash	Prevents DLL substitution attacks
5	Apply code obfuscation or packer to DLL payload	Obscures XOR logic from static view

11. Conclusion

Lab05 demonstrates how a two-stage binary protection scheme can be systematically dismantled using static disassembly and targeted tooling. Stage 1 was resolved via key recovery from Lab04. Stage 2 required identifying an XOR-encoded hex constant in the DLL disassembly and decoding it using hexor — a purpose-built Crystal tool that brute-forces all single-byte XOR candidates and filters for readable output.

The key insight is that XOR encoding, while better than plaintext, provides negligible security against an analyst with a brute force tool. The 255-candidate keyspace is exhausted in a single pass, and the printability filter eliminates false positives automatically.

- ALC-01 (High) — XOR-encoded key stored as movabs constant in .text — extractable via static disassembly.
- ALC-02 (Critical) — Single-byte XOR with 255-candidate keyspace decoded instantly by hexor --brute.
- ALC-03 (High) — No anti-debug or obfuscation in DLL — XOR loop and constant fully visible statically.

Lab Progression: Lab01 required runtime patching. Lab02 needed XOR inversion. Lab03 required modular arithmetic. Lab04 introduced binary patching. Lab05 introduces multi-stage analysis and custom tooling. Flags captured: FLAG{STAGE1_PASS} and FLAG{STAGE2_DLL_PASS}.

12. Appendix — Tools & References

Tool / Concept	Purpose
file	Binary format identification (PE32+, arch, sections)
wine	Execute Windows PE binary on Linux
r2 / Radare2	Static analysis framework — disassembly of loader.exe + payload.dll

Tool / Concept	Purpose
r2: afl	Function listing — sym.payload.dll_run (150 instr)
r2: pdf @ addr	Full function disassembly
r2: s addr	Seek (navigate) to a specific address
hexor	Custom Crystal-language XOR brute force decoder
hexor: --brute	Iterate all 255 single-byte XOR keys, filter printable results
hexor: parse_hex()	Convert hex string to little-endian byte array
hexor: readable?()	Printability filter — 85%+ printable ASCII threshold
movabs rax, 0x335157414d5b53	XOR-encoded DLL key constant @ 0x1db60140c
XOR key 0x12	Decoded AI_SEC! from 0x335157414d5b53 (little-endian)
FLAG{STAGE1_PASS}	Stage 1 flag — captured with key123
FLAG{STAGE2_DLL_PASS}	Stage 2 flag — captured with AI_SEC!

End of Report · SafeTest-Dev | Binary Security Research | Michael.A | April 2026

LAB05 — COMPLETED · FLAG{STAGE1_PASS} + FLAG{STAGE2_DLL_PASS}